AGLC CRM Phase 1 Implementation

GitHub + Notion + Development Infrastructure Setup Guide

Document Date: March 13, 2026
Status: Implementation Ready
Timeline: 8 weeks to MVP
Budget: SAR 90K - 120K

EXECUTIVE OVERVIEW

This document provides a complete setup guide for implementing the AGLC CRM Phase 1 using:

- **GitHub:** Version control, CI/CD, AI agent development, code review workflows
- **Notion:** Project management, documentation, team collaboration, knowledge base
- **Development Stack:** React 19 + Express 4 + tRPC 11 + PostgreSQL + Docker
- **Integration Tools:** DocuSign, Saudi Business Setup Platform, Calendly, SendGrid, Twilio

---

PART 1: GITHUB REPOSITORY STRUCTURE

1.1 Main Repository: `aglc-crm`

Purpose: Core CRM application (React frontend + Express backend)

Repository Structure:

```
aglc-crm/
├── client/                          # React 19 frontend
│   ├── src/
│   │   ├── pages/
│   │   │   ├── ProspectiveClient/   # PC module UI
│   │   │   ├── ClientManagement/    # Client management UI
│   │   │   ├── FeeProposals/        # Fee proposal UI
│   │   │   ├── Billing/             # Billing & invoicing UI
│   │   │   ├── TaskManagement/      # Task management UI
│   │   │   ├── Calendar/            # Calendar UI
│   │   │   ├── Documents/           # Document management UI
│   │   │   ├── ServiceRequests/     # Service requests UI
│   │   │   ├── Notifications/       # Notifications UI
│   │   │   ├── ExecutiveDashboard/  # Executive dashboard UI
│   │   │   ├── Reporting/           # Reporting UI
│   │   │   ├── ClientPortal/        # Client portal UI
│   │   │   ├── UserManagement/      # User management UI
│   │   │   └── ActivityLog/         # Activity log UI
│   │   ├── components/
│   │   │   ├── ui/                  # shadcn/ui components
│   │   │   ├── layouts/             # Page layouts
│   │   │   ├── forms/               # Reusable forms
│   │   │   └── charts/              # Chart components
│   │   ├── hooks/                   # Custom React hooks
│   │   ├── contexts/                # React contexts
│   │   ├── lib/
│   │   │   ├── trpc.ts              # tRPC client
│   │   │   └── utils.ts             # Utility functions
│   │   ├── App.tsx                  # Main app component
│   │   ├── main.tsx                 # Entry point
│   │   └── index.css                # Global styles
│   ├── public/                      # Static assets (favicon, robots.txt
only)
│   └── package.json
│
├── server/                          # Express backend
│   ├── routers/
│   │   ├── prospectiveClient.ts     # PC module procedures
│   │   ├── clientManagement.ts      # Client management procedures
│   │   ├── feeProposals.ts          # Fee proposal procedures
│   │   ├── billing.ts               # Billing procedures
│   │   ├── taskManagement.ts        # Task management procedures
│   │   ├── calendar.ts              # Calendar procedures
│   │   ├── documents.ts             # Document management procedures
│   │   ├── serviceRequests.ts       # Service request procedures
```

```
│   │   ├── notifications.ts         # Notification procedures
│   │   ├── executiveDashboard.ts    # Executive dashboard procedures
│   │   ├── reporting.ts             # Reporting procedures
│   │   ├── userManagement.ts        # User management procedures
│   │   ├── activityLog.ts           # Activity log procedures
│   │   └── auth.ts                  # Authentication procedures
│   ├── agents/                      # AI agent implementations
│   │   ├── briefingWizard.ts        # AI Briefing Wizard
│   │   ├── legalAssistant.ts        # AI Legal Assistant
│   │   ├── translator.ts            # Translation tool
│   │   └── businessCardScanner.ts   # Business card scanner
│   ├── services/                    # Business logic services
│   │   ├── prospectiveClientService.ts
│   │   ├── feeProposalService.ts
│   │   ├── billingService.ts
│   │   ├── notificationService.ts
│   │   └── ... (one per module)
│   ├── integrations/                # External service integrations
│   │   ├── docusign.ts              # DocuSign integration
│   │   ├── platform.ts              # Saudi Business Setup Platform
│   │   ├── calendly.ts              # Calendly integration
│   │   ├── sendgrid.ts              # Email service
│   │   ├── twilio.ts                # SMS/WhatsApp service
│   │   └── llm.ts                   # LLM service
│   ├── middleware/                  # Express middleware
│   │   ├── auth.ts                  # Authentication middleware
│   │   ├── errorHandler.ts          # Error handling
│   │   ├── logging.ts               # Request logging
│   │   └── rateLimit.ts             # Rate limiting
│   ├── db.ts                        # Database queries
│   ├── routers.ts                   # Main router
│   └── _core/                       # Framework core (do not modify)
│
├── drizzle/                         # Database schema
│   ├── schema.ts                    # All table definitions
│   ├── migrations/                  # Database migrations
│   └── seed.ts                      # Database seeding
│
├── shared/                          # Shared types and constants
│   ├── types.ts                     # Shared TypeScript types
│   ├── constants.ts                 # Shared constants
│   └── validators.ts                # Zod validators
│
├── tests/                           # Test files
│   ├── unit/                        # Unit tests
│   ├── integration/                 # Integration tests
```

```
│   ├── e2e/                        # End-to-end tests
│   └── fixtures/                   # Test data
│
├── docs/                           # Documentation
│   ├── API.md                      # API documentation
│   ├── ARCHITECTURE.md             # Architecture documentation
│   ├── DATABASE.md                 # Database documentation
│   ├── MODULES.md                  # Module documentation
│   └── DEPLOYMENT.md               # Deployment guide
│
├── .github/
│   ├── workflows/
│   │   ├── ci.yml                  # CI/CD pipeline
│   │   ├── tests.yml               # Automated testing
│   │   ├── security.yml            # Security scanning
│   │   └── deploy.yml              # Deployment workflow
│   └── ISSUE_TEMPLATE/             # Issue templates
│
├── docker-compose.yml              # Docker compose for local dev
├── Dockerfile                      # Production Docker image
├── .env.example                    # Environment variables template
├── package.json                    # Dependencies
├── tsconfig.json                   # TypeScript config
├── vitest.config.ts                # Test configuration
└── README.md                       # Project README
```

1.2 AI Agents Repository: `aglc-crm-ai-agents`

Purpose: Specialized AI agents for legal briefing, research, translation, and analysis

Repository Structure:

```
aglc-crm-ai-agents/
├── agents/
│   ├── briefing-wizard/            # AI Briefing Wizard agent
│   │   ├── src/
│   │   │   ├── index.ts            # Main agent implementation
│   │   │   ├── prompts.ts          # System prompts
│   │   │   ├── validators.ts       # Output validation
│   │   │   └── tests/
│   │   ├── package.json
│   │   └── README.md
│   ├── legal-assistant/            # AI Legal Assistant agent
│   │   ├── src/
│   │   │   ├── index.ts
│   │   │   ├── prompts.ts
│   │   │   ├── validators.ts
│   │   │   └── tests/
│   │   ├── package.json
│   │   └── README.md
│   ├── translator/                 # Translation agent
│   │   ├── src/
│   │   │   ├── index.ts
│   │   │   ├── prompts.ts
│   │   │   ├── validators.ts
│   │   │   └── tests/
│   │   ├── package.json
│   │   └── README.md
│   └── business-card-scanner/      # Business card scanner agent
│       ├── src/
│       │   ├── index.ts
│       │   ├── prompts.ts
│       │   ├── validators.ts
│       │   └── tests/
│       ├── package.json
│       └── README.md
│
├── orchestration/                  # Agent orchestration engine
│   ├── src/
│   │   ├── index.ts                # Main orchestrator
│   │   ├── queue.ts                # Task queue
│   │   ├── monitoring.ts           # Agent monitoring
│   │   └── tests/
│   ├── package.json
│   └── README.md
│
├── shared/                         # Shared agent utilities
```

```
│   ├── src/
│   │   ├── types.ts                # Agent types
│   │   ├── llm.ts                  # LLM interface
│   │   ├── validation.ts           # Validation utilities
│   │   └── logging.ts              # Logging utilities
│   ├── package.json
│   └── README.md
│
├── docs/
│   ├── ARCHITECTURE.md             # Agent architecture
│   ├── AGENTS.md                   # Agent specifications
│   ├── PROMPTS.md                  # Prompt engineering guide
│   └── TESTING.md                  # Testing guide
│
├── .github/
│   ├── workflows/
│   │   ├── test-agents.yml         # Agent testing workflow
│   │   ├── validate-prompts.yml    # Prompt validation
│   │   └── deploy-agents.yml       # Agent deployment
│   └── ISSUE_TEMPLATE/
│
├── docker-compose.yml              # Docker compose for testing
├── package.json                    # Root dependencies
└── README.md                       # Project README
```

1.3 Documentation Repository: `aglc-crm-docs`

Purpose: Comprehensive documentation, guides, and knowledge base

Repository Structure:

```
aglc-crm-docs/
├── guides/
│   ├── GETTING_STARTED.md           # Getting started guide
│   ├── ARCHITECTURE.md              # System architecture
│   ├── DATABASE.md                  # Database guide
│   ├── API.md                       # API documentation
│   ├── MODULES.md                   # Module documentation
│   ├── DEPLOYMENT.md                # Deployment guide
│   ├── SECURITY.md                  # Security guide
│   └── TROUBLESHOOTING.md           # Troubleshooting guide
│
├── workflows/
│   ├── PC_MODULE_WORKFLOW.md        # PC module workflow
│   ├── FEE_PROPOSAL_WORKFLOW.md     # Fee proposal workflow
│   ├── CASE_WORKFLOW.md             # Case workflow
│   ├── BILLING_WORKFLOW.md          # Billing workflow
│   └── EXECUTIVE_WORKFLOW.md        # Executive workflow
│
├── specifications/
│   ├── MODULE_SPECIFICATIONS.md     # All 22 modules detailed
│   ├── DATA_STRUCTURES.md           # Data structure specifications
│   ├── API_SPECIFICATIONS.md        # API specifications
│   ├── PERMISSION_MATRIX.md         # Role-based permissions
│   └── INTEGRATION_SPECS.md         # Integration specifications
│
├── team/
│   ├── ROLES_AND_RESPONSIBILITIES.md # Team roles
│   ├── ONBOARDING.md                # Team onboarding guide
│   ├── DEVELOPMENT_STANDARDS.md     # Development standards
│   └── CODE_REVIEW_GUIDELINES.md    # Code review guidelines
│
├── operations/
│   ├── MAINTENANCE.md               # Maintenance procedures
│   ├── MONITORING.md                # Monitoring and alerts
│   ├── BACKUP_RECOVERY.md           # Backup and recovery
│   ├── SCALING.md                   # Scaling guide
│   └── PERFORMANCE_TUNING.md        # Performance tuning
│
├── images/                          # Documentation images
│   ├── architecture-diagram.png
│   ├── workflow-diagrams/
│   ├── ui-mockups/
│   └── database-schema.png
│
├── .github/
```

```
│   └── workflows/
│       └── validate-docs.yml        # Documentation validation
│
└── README.md                        # Documentation index
```

PART 2: NOTION WORKSPACE STRUCTURE

2.1 Notion Workspace Setup

Main Workspace: AGLC CRM Development

Pages and Databases:

1. Project Management

- **Database: Sprint Planning**
  - Sprint number, start date, end date, goals, status
  - Linked to tasks and team members
- **Database: Tasks**
  - Task name, description, assignee, sprint, status, priority, due date
  - Linked to modules, team members, and dependencies
- **Database: Bugs & Issues**
  - Bug title, description, severity, status, assignee, module
  - Linked to tasks and pull requests
- **Database: Features**
  - Feature name, description, module, status, priority, owner
  - Linked to tasks and documentation

2. Module Documentation

- **Database: Modules (22 total)**
  - Module name, description, status, owner, team members
  - Features, API endpoints, database tables
  - Testing status, documentation status
  - Linked to tasks, bugs, and pull requests
- **Pages:**
  - PC Module (Prospective Client)
  - Client Management Module
  - Fee Proposals Module
  - Billing & Invoicing Module
  - Task Management Module
  - Calendar Module
  - Document Management Module
  - Service Requests Module
  - Notifications Module
  - Executive Dashboard Module
  - Reporting Module
  - Client Portal Module
  - User Management Module
  - Activity Log Module
  - AI Briefing Wizard Module
  - AI Legal Assistant Module
  - Translation Tool Module
  - Business Card Scanner Module
  - Timesheets Module
  - HR Module
  - System Configuration Module

3. Team & Roles

- **Database: Team Members**
  - Name, role, email, phone, GitHub username, Notion profile
  - Assigned modules, skills, availability
  - Linked to tasks and pull requests
- **Database: Roles & Permissions**
  - Role name, description, permissions
  - Modules access, features access
  - Linked to team members

4. Integration & External Services

- **Database: Integrations**
  - Service name, status, credentials location, documentation
  - Integration owner, testing status
  - Linked to modules
- **Pages:**
  - DocuSign Integration
  - Saudi Business Setup Platform Integration
  - Calendly Integration
  - SendGrid Integration
  - Twilio Integration
  - LLM Service Integration

5. Knowledge Base

- **Database: Documentation**
  - Document title, category, status, author, last updated
  - Linked to modules and team members

- **Pages:**
  - Architecture Overview
  - Database Schema
  - API Documentation
  - Development Standards
  - Code Review Guidelines
  - Security Guidelines
  - Deployment Procedures

6. Timeline & Roadmap

- **Database: Roadmap**
  - Phase, week, features, status, owner
  - Linked to tasks and modules
- **Calendar View:**
  - Sprint timeline
  - Milestone dates
  - Deployment dates
  - Team availability

7. Metrics & Analytics

- **Database: Metrics**
  - Metric name, current value, target value, category
  - Last updated, owner
  - Linked to modules
- **Pages:**
  - Development Velocity
  - Code Quality Metrics
  - Test Coverage

- Performance Metrics
- User Adoption Metrics

PART 3: GITHUB WORKFLOWS & CI/CD

3.1 GitHub Actions Workflows

Workflow 1: Continuous Integration (ci.yml)

```yaml
name: CI Pipeline

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main, develop ]

jobs:
  test:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_PASSWORD: postgres
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
        ports:
          - 5432:5432

    steps:
      - uses: actions/checkout@v3

      - uses: actions/setup-node@v3
        with:
          node-version: '22'
          cache: 'pnpm'

      - run: pnpm install

      - run: pnpm run check
      - run: pnpm run format --check
```

```yaml
      - run: pnpm run test
      - run: pnpm run build

      - name: Upload coverage
        uses: codecov/codecov-action@v3
        with:
          files: ./coverage/coverage-final.json
```

Workflow 2: Security Scanning (security.yml)

```yaml
name: Security Scanning

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main, develop ]
  schedule:
    - cron: '0 0 * * 0'  # Weekly

jobs:
  security:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - uses: actions/setup-node@v3
        with:
          node-version: '22'
          cache: 'pnpm'

      - run: pnpm install

      - name: Run security audit
        run: pnpm audit --audit-level=moderate

      - name: SAST scanning
        uses: github/super-linter@v4
        env:
          DEFAULT_BRANCH: main
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}

      - name: Dependency check
        uses: dependency-check/Dependency-Check_Action@main
        with:
          project: 'aglc-crm'
          path: '.'
          format: 'JSON'
```

Workflow 3: Deployment (deploy.yml)

```yaml
name: Deploy to Production

on:
  push:
    branches: [ main ]
    paths:
      - 'server/**'
      - 'client/**'
      - 'drizzle/**'
      - 'package.json'

jobs:
  deploy:
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'

    steps:
      - uses: actions/checkout@v3

      - uses: actions/setup-node@v3
        with:
          node-version: '22'
          cache: 'pnpm'

      - run: pnpm install
      - run: pnpm run build

      - name: Deploy to production
        run: |
          # Deploy commands here
          echo "Deploying to production..."
```

PART 4: DEVELOPMENT SETUP

4.1 Local Development Environment

Prerequisites:

- Node.js 22.13.0

- pnpm 10.4.1
- PostgreSQL 15
- Docker & Docker Compose
- Git

Setup Steps:

```
# Clone the main repository
gh repo clone aglc/aglc-crm
cd aglc-crm

# Install dependencies
pnpm install

# Copy environment variables
cp .env.example .env.local

# Start PostgreSQL with Docker
docker-compose up -d postgres

# Run database migrations
pnpm db:push

# Start development server
pnpm dev

# In another terminal, start tests in watch mode
pnpm test --watch
```

Environment Variables (.env.local):

```
# Database
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/aglc_crm_dev

# OAuth
VITE_APP_ID=your_app_id
OAUTH_SERVER_URL=https://api.manus.im
VITE_OAUTH_PORTAL_URL=https://portal.manus.im

# External Services
DOCUSIGN_API_KEY=your_docusign_key
DOCUSIGN_ACCOUNT_ID=your_account_id
PLATFORM_API_KEY=your_platform_key
CALENDLY_API_KEY=your_calendly_key
SENDGRID_API_KEY=your_sendgrid_key
TWILIO_ACCOUNT_SID=your_twilio_sid
TWILIO_AUTH_TOKEN=your_twilio_token
LLM_API_KEY=your_llm_key

# Application
JWT_SECRET=your_jwt_secret
NODE_ENV=development
```

4.2 Code Quality Standards

Linting & Formatting:

```
# Format code
pnpm run format

# Check formatting
pnpm run format --check

# Lint code
pnpm run lint

# Type checking
pnpm run check
```

Testing:

```
# Run all tests
pnpm test

# Run tests in watch mode
pnpm test --watch

# Run tests with coverage
pnpm test --coverage

# Run specific test file
pnpm test server/routers/prospectiveClient.test.ts
```

PART 5: IMPLEMENTATION TIMELINE

Phase 1: Weeks 1-8 (MVP)

Week 1-2: Foundation & Infrastructure

- ✅ GitHub repositories setup
- ✅ Notion workspace setup
- ✅ Database schema design
- ✅ API specifications
- ✅ Development environment setup
- ✅ CI/CD pipeline configuration

Week 3-4: Core Modules

- ✅ Client Management Module
- ✅ Task Management Module
- ✅ Calendar Module
- ✅ Service Requests Module
- ✅ Document Management Module

Week 5: Revenue Modules

- ✅ Fee Proposals Module (all 7 categories)
- ✅ Billing & Invoicing Module
- ✅ Timesheets Module

Week 6: Client Lifecycle

- ✅ Prospective Client (PC) Module
- ✅ Client Portal Module
- ✅ Notifications Module

Week 7: Intelligence & Executive

- ✅ AI Briefing Wizard
- ✅ AI Legal Assistant
- ✅ Executive Dashboard Module
- ✅ Reporting Module

Week 8: Testing & Deployment

- ✅ Integration testing
- ✅ Security testing
- ✅ Performance testing
- ✅ Documentation finalization
- ✅ Production deployment

PART 6: TEAM ASSIGNMENTS

Development Team

Role	Name	GitHub	Notion	Modules
Tech Lead	[TBD]	@tech-lead	@tech-lead	Architecture, Infrastructure
Backend Lead	[TBD]	@backend-lead	@backend-lead	Server, APIs, Database
Frontend Lead	[TBD]	@frontend-lead	@frontend-lead	Client, UI/UX
AI/Agents Lead	[TBD]	@ai-lead	@ai-lead	AI agents, Orchestration
QA Lead	[TBD]	@qa-lead	@qa-lead	Testing, Quality Assurance
DevOps Lead	[TBD]	@devops-lead	@devops-lead	CI/CD, Deployment, Infrastructure
Product Manager	[TBD]	@product-manager	@product-manager	Requirements, Roadmap

---

PART 7: INTEGRATION TOOLS & SERVICES

7.1 External Service Integrations

Service	Purpose	Status	Owner	Documentation
DocuSign	Electronic signature for fee proposals	⏳ To Implement	Backend Lead	[Link]
Saudi Business Setup Platform	PC module intake automation	⏳ To Implement	Backend Lead	[Link]
Calendly	Calendar integration	⏳ To Implement	Backend Lead	[Link]
SendGrid	Email notifications	⏳ To Implement	Backend Lead	[Link]
Twilio	SMS/WhatsApp notifications	⏳ To Implement	Backend Lead	[Link]
OpenAI	LLM for AI agents	⏳ To Implement	AI Lead	[Link]
PostgreSQL	Database	✅ Ready	DevOps Lead	[Link]
Docker	Containerization	✅ Ready	DevOps Lead	[Link]
GitHub Actions	CI/CD	✅ Ready	DevOps Lead	[Link]

PART 8: SUCCESS METRICS

Development Metrics

Metric	Target	Current	Owner
Code Coverage	80%+	TBD	QA Lead
Test Pass Rate	100%	TBD	QA Lead
Build Time	< 5 min	TBD	DevOps Lead
Deployment Time	< 10 min	TBD	DevOps Lead
System Uptime	99.9%+	TBD	DevOps Lead
Response Time	< 500ms	TBD	Backend Lead
Page Load Time	< 2s	TBD	Frontend Lead

Business Metrics

Metric	Target	Current	Owner
Team Adoption	90%+	TBD	Product Manager
Feature Usage	80%+	TBD	Product Manager
Time Saved	60%+	TBD	Product Manager
Error Reduction	95%+	TBD	Product Manager
User Satisfaction	4.5/5	TBD	Product Manager
Task Completion	95%+ on-time	TBD	Product Manager

NEXT STEPS

1. **Create GitHub Organization** - Set up repositories
2. **Invite Team Members** - Grant appropriate access

3. **Create Notion Workspace** - Set up project management
4. **Configure Integrations** - Set up external services
5. **Begin Week 1 Tasks** - Start infrastructure setup
6. **Schedule Kickoff Meeting** - Align team on plan

Status: READY FOR IMPLEMENTATION
Timeline: 8 weeks to MVP
Budget: SAR 90K - 120K